PREMIER UNIVERSITY, CHATTOGRAM

PROJECT REPORT

Course Name	**Mobile Application Development**
Course Code	CSE 2210
Name of Report Topic	House Booking System
Number of Report Topic	01
Date of Submission	**10.05.2026**

Submitted by (Name - Roll No.)	**<u>Submitted To:</u> Rashed Miah**	
Thanjilur Rahman Sayeed - 0222320005101162	BATCH: 44	SEMESTER: 5TH
Urmi Das - 0222320005101164	SESSION: FALL – 25	
Ifty Anwar - 0222320005101192	GROUP NAME : TEAM VISIONARIES	
Al Amin - 0222320005101200	SECTION: E	**REMARKS :**

Contents

01. Title Page

Project Title: House Booking System

02. Abstract

The MAD Project House Booking System is a comprehensive mobile application developed to modernize the traditional house rental process. Unlike conventional platforms that only provide property listings, this system introduces a structured booking mechanism, role-based access control and centralized administrative management.

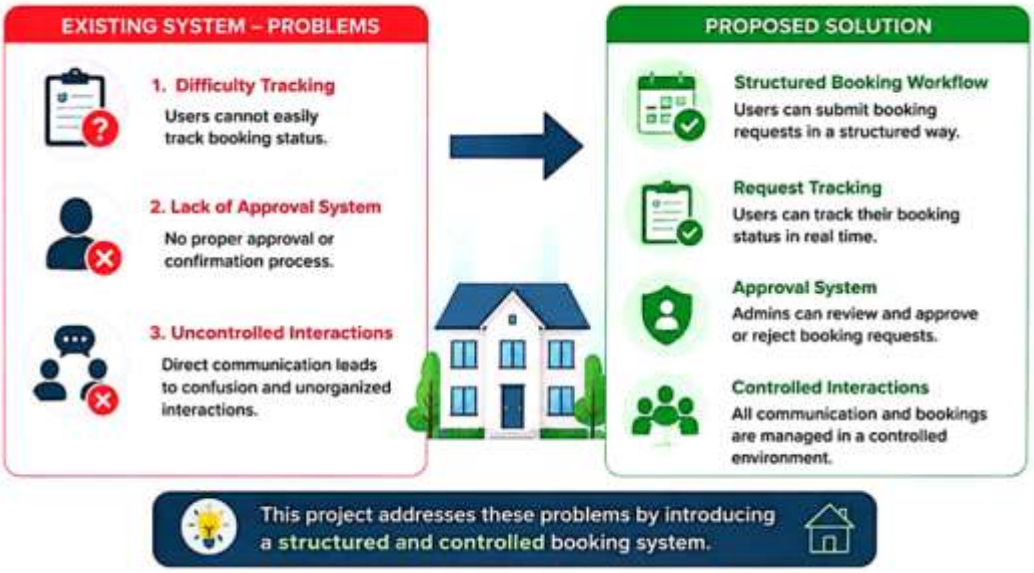
The application allows users to explore properties, submit booking requests with scheduling preferences and track the status of their requests in real time. Administrators are given full control over property listings and booking approvals, ensuring a transparent and controlled environment. By integrating Firebase services, the system ensures secure authentication and efficient data handling. Overall, the project aims to deliver a more organized, reliable and user-friendly solution for house booking.

03. Problem Statement

Existing platforms like Tolet BD mainly focus on property listings and direct communication. However, they lack structured booking workflows, request tracking and administrative control.

Users face issues such as:

- Difficulty tracking booking status
- Lack of approval system
- Uncontrolled interactions



This project addresses these problems by introducing a structured and controlled booking system.

04. Objectives

The primary goal of this project is to design and develop a structured and user-friendly house booking system.

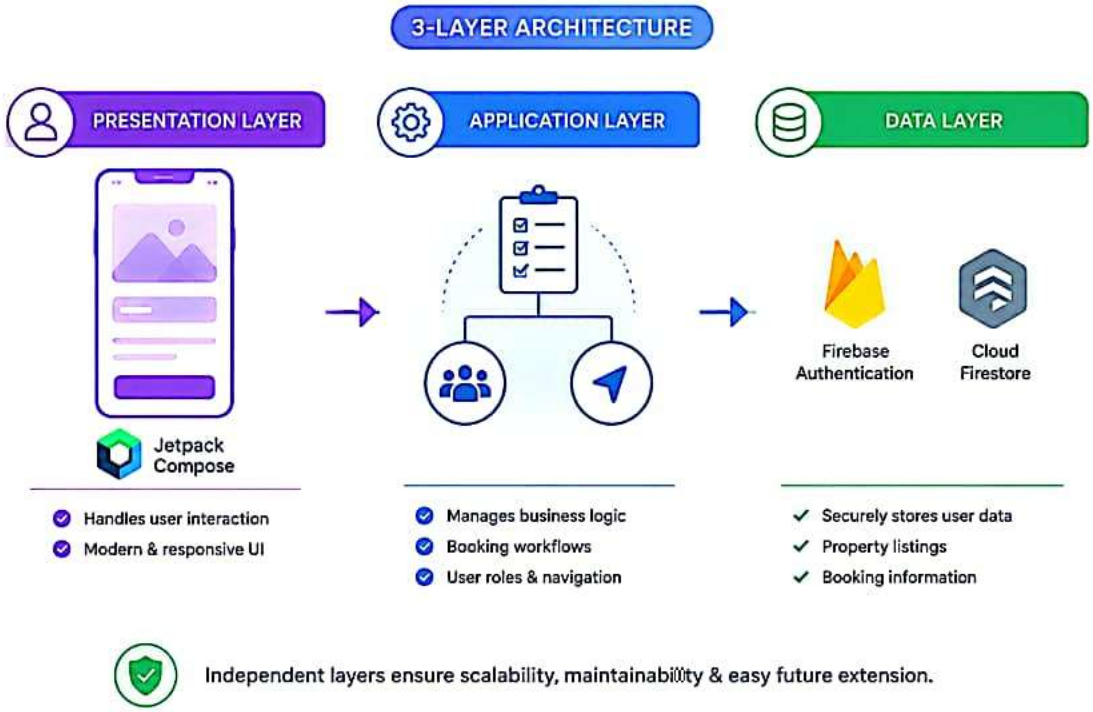
Specific objectives include:

- To implement a formal booking request system
- To provide role-based access for users, admins and guests
- To ensure transparency in booking and approval processes
- To improve overall user experience through organized features
- To enable efficient property management through admin control

05. System Architecture

The system is designed using three-layer architecture to ensure scalability and maintainability.

a) The presentation layer handles user interaction and is built using Jetpack Compose, providing a modern and responsive interface.
b) The application layer manages business logic, including booking workflows, user roles and navigation between screens.
c) The data layer is powered by Firebase Authentication and Cloud Firestore which securely store user data, property listings and booking information.



This layered approach ensures that each component of the system operates independently, making the application easier to maintain and extend in the future.

06. System Requirements

6.1 Functional Requirements

- User/Admin authentication
- Guest browsing mode
- Property browsing & filtering

- Booking request submission
- Favorites management
- Reviews and comments
- Admin property management
- Booking approval system

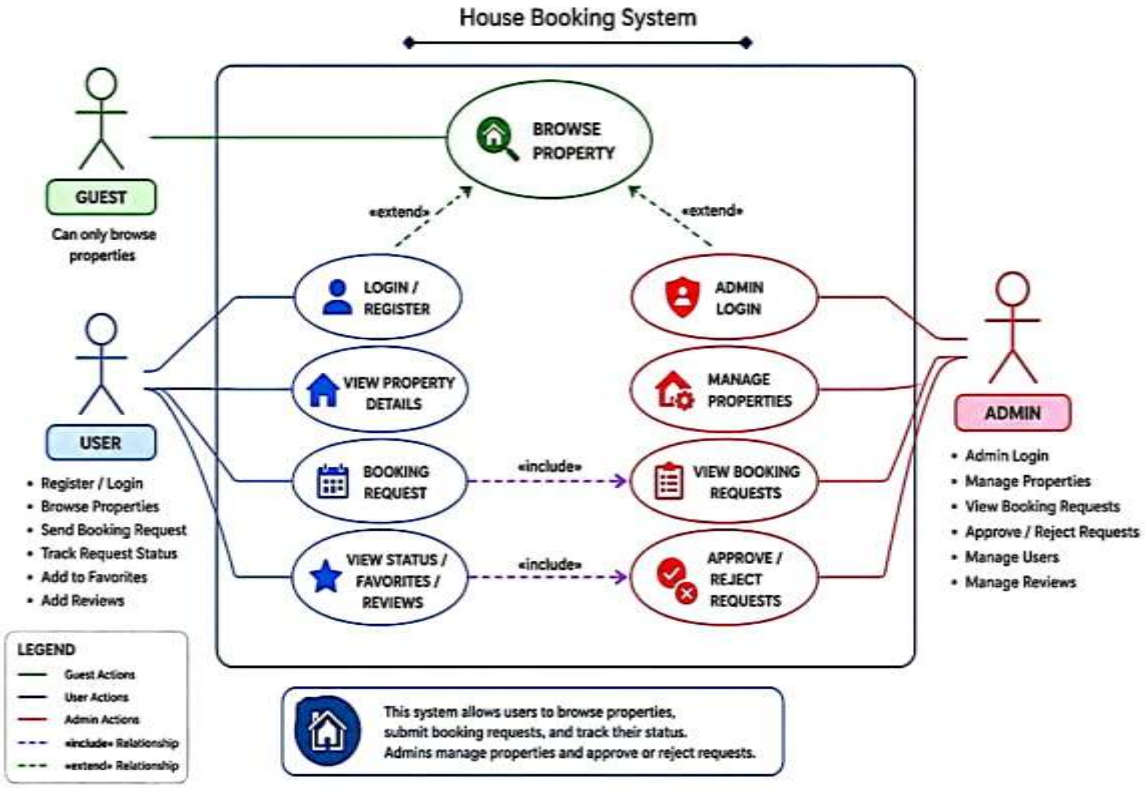
6.2 Non-Functional Requirements

- Secure authentication
- Role-based access control
- Scalable architecture
- User-friendly UI

07. System Features

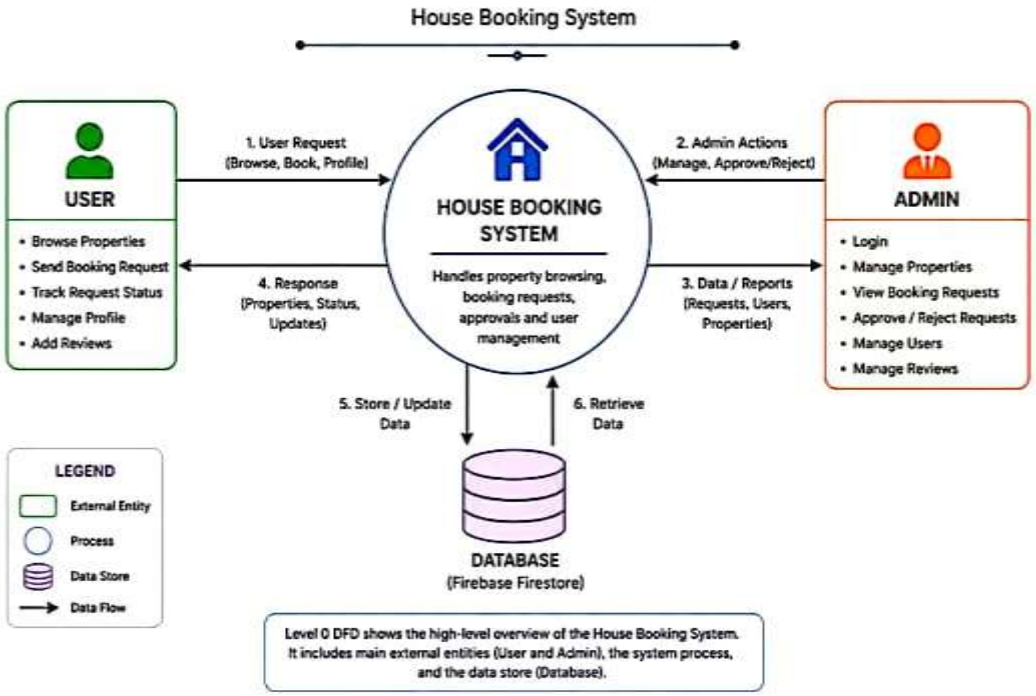
- Authentication & Access Modes
- Property Browsing & Filtering
- Property Details View
- Booking Workflow
- Favorites System
- Reviews & Comments
- Property Request System
- Profile Tracking
- Admin Control Panel

08. Use Case Diagram

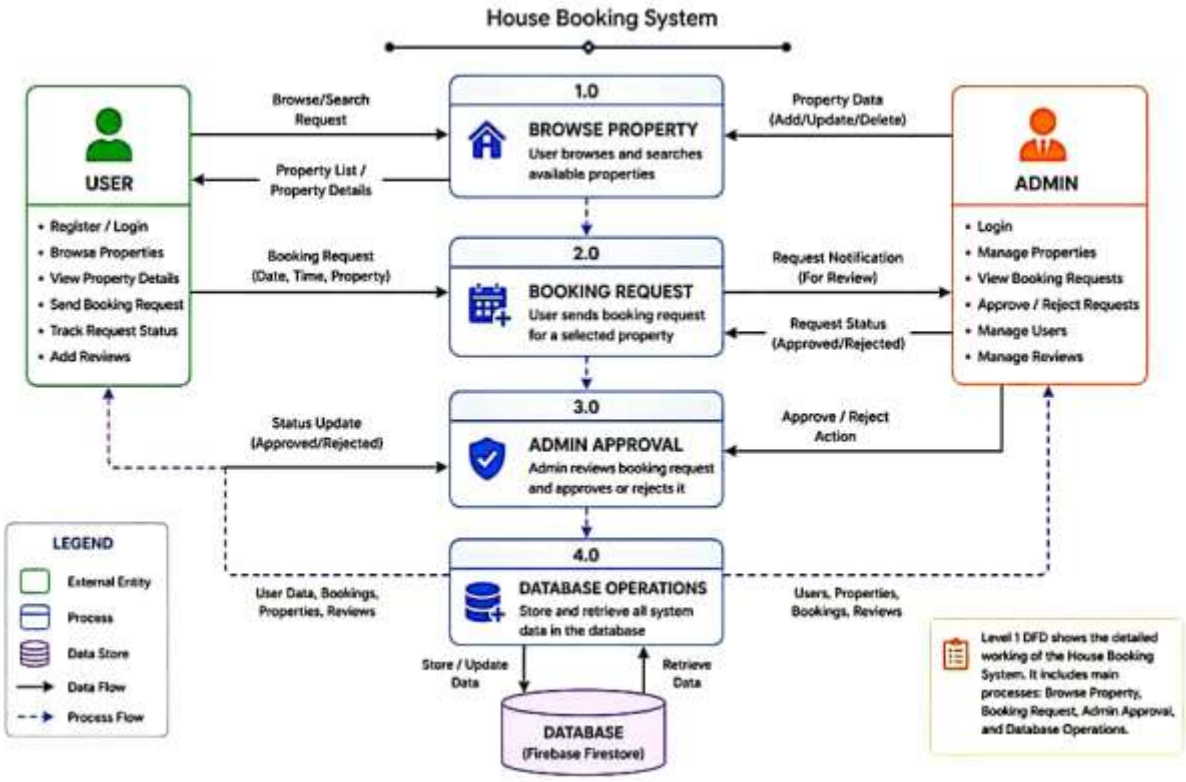


09. Data Flow Diagram (DFD)

a) **Level 0 :**



b) Level 1:



10. Database Design

The system uses Cloud Firestore as its database, organized into multiple collections.

a) The Users collection stores user information such as name, email and role.
b) The Properties collection contains details about each property, including category, price and images.
c) The Bookings collection records booking requests along with their status.
d) The Reviews collection stores user feedback and ratings.

This structured design ensures efficient data management and quick retrieval of information.

11. Security Features

- Firebase Authentication
- Firestore access rules
- Role-based authorization

12. Methodology

The project follows a modular development methodology. Each feature, such as authentication, booking and admin management is developed independently and later integrated into the main system.

This approach allows easier debugging, testing and future expansion. Agile practices were followed to ensure continuous improvement during development.

13. Technology Used

- Kotlin
- Jetpack Compose
- Firebase Authentication
- Cloud Firestore
- Android Studio

14. Implementation

The system is implemented using Kotlin as the programming language and Jetpack Compose for UI design. Firebase Authentication handles login functionality while Firestore manages data storage.

Different screens and modules are connected using Navigation Compose, ensuring smooth transitions and user experience.

15. Result & Discussion

The developed system successfully addresses the limitations of traditional house rental platforms. It provides a structured workflow, better control and improved transparency.

Users can easily track their booking status while admins can efficiently manage requests and listings. This results in a more organized and user-friendly system.

16. System Interface (Screenshots Section)

Figure 1: Login Screen

This screen allows users to securely log in using their email and password. It ensures proper authentication and controls access to user and admin features.

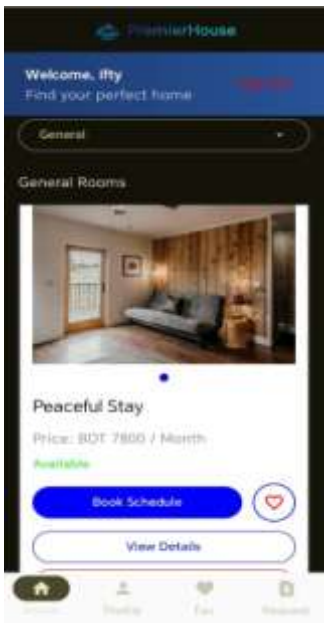


Figure 2: Home Screen

The home screen displays a list of available properties with brief details. Users can easily browse listings and navigate to view more detailed information.

Figure 3: Property Details

This screen provides complete information about a selected property. It includes images, pricing, location and options to proceed with booking.

Figure 4: Booking Request Page

Users can submit a booking request by selecting preferred date and time.The system processes the request and sends it for confirmation or approval.

Figure 5: Admin Panel

The admin panel allows administrators to manage users, properties and bookings. It provides centralized control for monitoring and maintaining system operations.

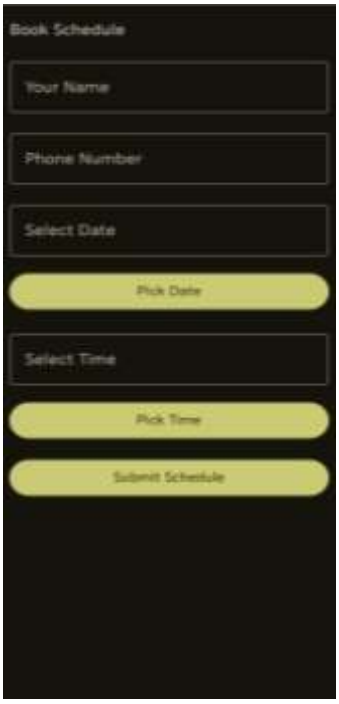


Figure 6: Book schedule

This screen shows all scheduled bookings in a structured format. It helps both users and admin to track booking timelines and availability easily.

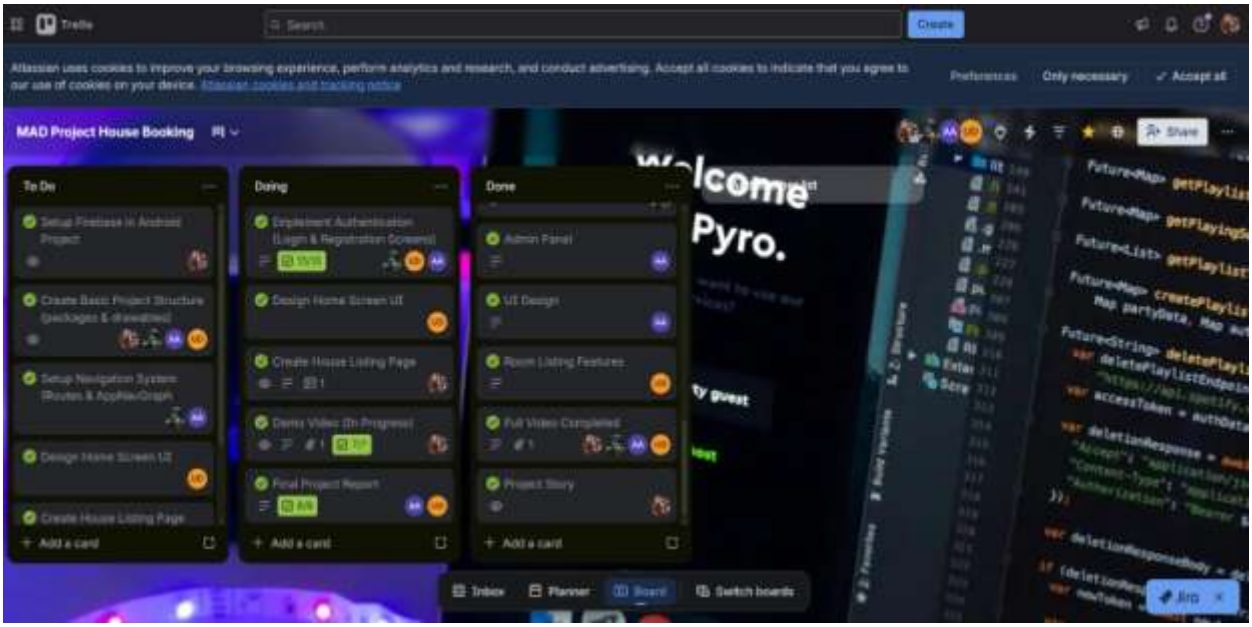
17. Project Management Using Trello

Overview:

To efficiently manage the development process, the team used Trello for organizing tasks and tracking progress.

Workflow Structure:

- **To Do:** Planned tasks
- **In Progress:** Ongoing tasks
- **Completed:** Finished tasks



Task Distribution:

Tasks were divided among team members:

- UI Design
- Firebase Integration
- Booking System
- Admin Panel

Progress Tracking:

Trello helped monitor:

- Task completion
- Deadlines
- Team contributions

Benefits:

- Better collaboration
- Organized workflow
- Improved productivity

18. Core Implementation Code Snippets

1. Login.kt:

Handles user authentication and login process using secure credentials.

Code:

```
package com.example.house_booking.screens

import android.widget.Toast

import
androidx.compose.foundation.background

import androidx.compose.foundation.clickable

import androidx.compose.foundation.Image

import androidx.compose.foundation.layout.*

import
androidx.compose.foundation.shape.Rounded
CornerShape

import androidx.compose.material3.*

import androidx.compose.runtime.*

import
androidx.compose.runtime.livedata.observeAs
State

import androidx.compose.ui.Alignment

import androidx.compose.ui.Modifier

import androidx.compose.ui.graphics.Brush

import androidx.compose.ui.graphics.Color

import
androidx.compose.ui.platform.LocalContext

import androidx.compose.ui.text.TextStyle

import
androidx.compose.ui.text.font.FontWeight

import
androidx.compose.ui.text.style.TextDecoration

import androidx.compose.ui.unit.dp

import androidx.compose.ui.unit.sp

import
androidx.compose.ui.res.painterResource

import androidx.navigation.NavHostController

import
com.example.house_booking.data_util.AuthSt
ate

import
com.example.house_booking.data_util.AuthVi
ewModel

import com.example.house_booking.R

@Composable

fun LoginAdmin(navController:
NavHostController, authViewModel:
AuthViewModel) {

   var email by remember {
mutableStateOf("") }

   var password by remember {
mutableStateOf("") }

   val authState =
authViewModel.authState.observeAsState()

   val context = LocalContext.current

   LaunchedEffect(authState.value) {

      when (authState.value) {

         is AuthState.Authenticated ->
navController.navigate("adminLanding")
```

```
            is AuthState.Error -> Toast.makeText(
                context,
                (authState.value as
AuthState.Error).message,
                Toast.LENGTH_SHORT
            ).show()
            else -> Unit
        }
    }
    val gradientBrush = Brush.verticalGradient(
        colors = listOf(
            Color(0xFF2193b0),
            Color(0xFF6dd5ed)
        )
    )
    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(gradientBrush),
        contentAlignment = Alignment.Center
    ) {
        Card(
            shape = RoundedCornerShape(16.dp),
            colors =
CardDefaults.cardColors(containerColor =
Color.White.copy(alpha = 0.95f)),
            elevation =
CardDefaults.cardElevation(defaultElevation
= 10.dp),
            modifier = Modifier
                .fillMaxWidth(0.9f)
                .padding(16.dp)
        ) {
            Column(
                modifier = Modifier.padding(24.dp),
                verticalArrangement =
Arrangement.spacedBy(16.dp),
                horizontalAlignment =
Alignment.CenterHorizontally
            ) {
                Image(
                    painter = painterResource(id =
R.drawable.premier_logo),
                    contentDescription =
"PremierHouse Logo",
                    modifier = Modifier
                        .size(80.dp)
                        .padding(bottom = 8.dp)
                )
                Text(
                    text = "PremierHouse",
                    fontSize = 26.sp,
                    fontWeight = FontWeight.Bold,
                    style = TextStyle(
                        brush = Brush.linearGradient(
                            colors =
listOf(Color(0xFF1A2980),
Color(0xFF26D0CE))
```

```
                ),
            ),
            modifier =
Modifier.padding(bottom = 8.dp)
        )
        Divider(
            color = Color.LightGray,
            thickness = 1.dp,
            modifier =
Modifier.padding(vertical = 4.dp)
        )
        Text(
            text = "Admin Log In",
            fontSize = 20.sp,
            fontWeight =
FontWeight.Medium,
            color = Color.DarkGray,
            modifier =
Modifier.padding(bottom = 8.dp)
        )
        OutlinedTextField(
            value = email,
            onValueChange = { email = it },
            label = { Text("Email") },
            singleLine = true,
            modifier =
Modifier.fillMaxWidth()
        )
        OutlinedTextField(
            value = password,
            onValueChange = { password = it
},
            label = { Text("Password") },
            singleLine = true,
            modifier =
Modifier.fillMaxWidth()
        )
        Button(
            onClick = {
authViewModel.login(email, password) },
            modifier = Modifier
                .fillMaxWidth()
                .height(50.dp),
            shape =
RoundedCornerShape(12.dp),
            colors =
ButtonDefaults.buttonColors(containerColor =
Color(0xFF1A2980))
        ) {
            Text(
                "Login",
                style = TextStyle(
                    fontSize = 18.sp,
                    color = Color.White
                )
            )
        }
```

```
            Spacer(modifier =
Modifier.height(16.dp))

            Row(

                verticalAlignment =
Alignment.CenterVertically,

                horizontalArrangement =
Arrangement.Center,

                modifier =
Modifier.fillMaxWidth()

            ) {

                Text(text = "Are you a User? ")

                Spacer(modifier =
Modifier.width(4.dp))

                Text(

                    text = "User Login",

                    color =
MaterialTheme.colorScheme.primary,

                    textDecoration =
TextDecoration.Underline,

                    modifier = Modifier.clickable {


navController.navigate("login")

                    }

                )

            }

        }

    }

}
```

2. Rule.kt:

Defines validation rules and business logic to ensure correct data processing.

Code:

```
package com.example.house_booking.screens

import
androidx.compose.foundation.layout.Column

import
androidx.compose.foundation.layout.fillMaxSize

import
androidx.compose.foundation.layout.fillMaxWidth

import
androidx.compose.foundation.layout.padding

import
androidx.compose.foundation.lazy.LazyColumn

import androidx.compose.foundation.lazy.items

import
androidx.compose.foundation.shape.RoundedCorn
erShape

import androidx.compose.material3.Button

import androidx.compose.material3.ButtonDefaults

import androidx.compose.material3.Card

import androidx.compose.material3.CardDefaults

import androidx.compose.material3.MaterialTheme

import androidx.compose.material3.Text
```

```
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.navigation.NavHostController
@Composable
fun Rules(navController: NavHostController) {
   Column(
      modifier = Modifier
         .fillMaxSize()
         .padding(16.dp)
   ) {
      Text(
         text = "User Guidelines",
         style = MaterialTheme.typography.headlineLarge.copy(
            fontWeight = FontWeight.Bold,
            color = MaterialTheme.colorScheme.primary
         ),
         modifier = Modifier.padding(bottom = 16.dp)
      )
      Text(
         text = "Please take a moment to read our guidelines. These are designed to make your experience better and safer.",
         style = MaterialTheme.typography.bodyMedium.copy(color = Color.Gray),
         modifier = Modifier.padding(bottom = 24.dp)
      )
      LazyColumn(
         modifier = Modifier.weight(1f)
      ) {
         items(guidelines) { guideline ->
            GuidelineCard(guideline)
         }
      }
      Button(
         onClick = { navController.popBackStack() },
         modifier = Modifier
            .fillMaxWidth()
            .padding(top = 16.dp),
         shape = RoundedCornerShape(12.dp),
```

```
            colors = ButtonDefaults.buttonColors(
                containerColor = MaterialTheme.colorScheme.primary,
                contentColor = Color.White
            )
        ) {
            Text("I Agree", fontSize = 16.sp)
        }
    }
}

@Composable
fun GuidelineCard(guideline: String) {
    Card(
        shape = RoundedCornerShape(12.dp),
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 8.dp),
        colors = CardDefaults.cardColors(containerColor = Color.White)
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text(
                text = guideline,
                style = MaterialTheme.typography.bodyMedium.copy(color = Color.Black)
            )
        }
    }
}
```

3. NaveiItem.kt:

Manages navigation between different screens within the application.

Code:

```
package com.example.house_booking.data_util

import androidx.compose.ui.graphics.vector.ImageVector

data class NavItem(
    val label: String,
    val icon: ImageVector,
    val route: String    )
```

19. Advantages

- Structured booking system
- Easy tracking
- Better control
- User-friendly interface

20. Limitations

- No online payment system
- No real-time chat
- Limited offline support

21. Future Work

- Payment integration (bKash, Nagad)
- Real-time chat
- Google Maps integration
- AI recommendations
- Push notifications

22. Conclusion

The MAD Project House Booking System provides a modern solution to traditional house rental problems. By introducing structured workflows, role-based access and centralized management, the system ensures better organization and user satisfaction.

It offers a significant improvement over existing platforms and has strong potential for further development.

23. References

- Tolet BD Application
- Firebase Documentation
- Android Developer Documentation
- Jetpack Compose Guide